Enterprise Architecture Design — Supplemental Assignment

Module: ENTPH6001 Enterprise Architecture Design

Submission: Cloud-native architecture for a small e-commerce checkout platform

Word count (Sections 1–5, excluding figures, references and appendices): 1993

Repository: https://github.com/TravellerXi/entph6001-checkout-platform

Screencast: `screencast.mp4`, submitted alongside this report (see `docs/screencast-script.md`)

---

1. Architecture Proposal

The organisation runs its checkout application as a small set of containers, leaving every dependency implicit: anything can call anything, configuration travels with the image, and one slow component stalls the customer path. The proposal makes these relationships enforceable.

Service decomposition. The system is decomposed along business capability rather than technical layer, following bounded-context reasoning: `pricing-fn` owns tax and totals, `inventory-fn` owns stock and the stock datastore, and `checkout-fn` composes them into an order decision. A separate `gateway` serves the UI and exposes one API route. The split is deliberately small: at this size the dominant risk is operational overhead, not team scaling, so services were justified by differing rates of change and failure isolation rather than by count. `inventory-fn` alone owns its schema, preventing the shared-database coupling that turns a distributed system into a distributed monolith.

Request entry. External traffic enters through one controlled point: a Traefik Ingress routing to `gateway-svc`. The gateway terminates the public contract, exposes only `/api/checkout`, and either honours an inbound `X-Request-ID` or mints one. Internal services publish no external route.

Trust tiers. Workloads are classified **public** (gateway), **internal** (checkout, pricing, inventory) and **protected** (PostgreSQL). The classification is not documentation: a deny-by-default NetworkPolicy covering **both ingress and egress** (Kubernetes, 2026b), plus directed allow-rules, makes each tier boundary a control the platform enforces. Omitting egress rules is a serious gap: a compromised workload with unrestricted outbound access can exfiltrate data or reach a metadata endpoint, the mechanism behind the Capital One breach (Lee, 2026). Here `pricing-fn` and `postgres` have empty egress, and only `inventory-fn` reaches the database.

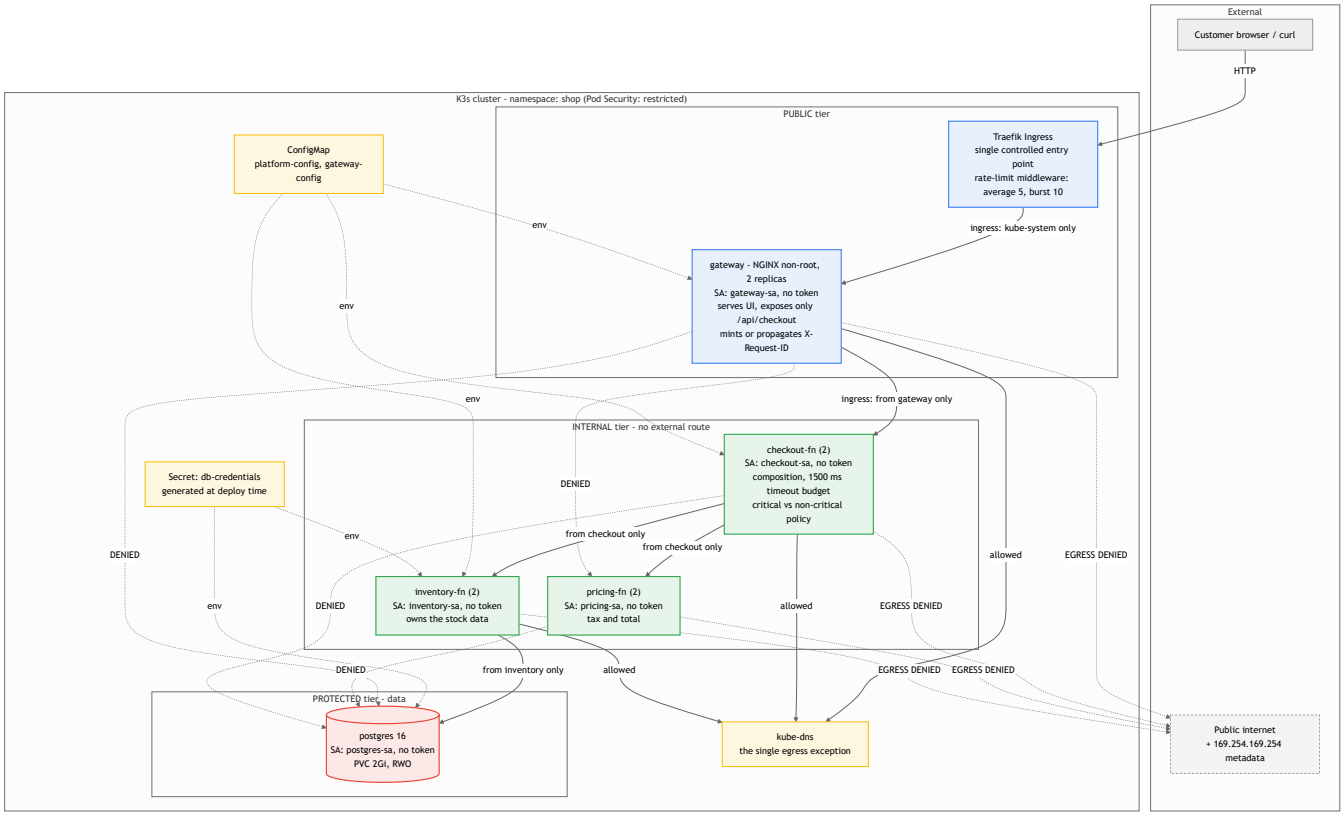
State, configuration and secrets. Stock data lives in PostgreSQL backed by a PersistentVolumeClaim. Non-secret configuration (dependency URLs, timeout budget, tax rate, database host) comes from a ConfigMap; credentials come from a Secret injected as environment variables. Nothing is baked into an image, and the repository holds only a Secret *template*, the real value generated at deploy time.

Workload identity. Each workload runs under its own ServiceAccount holding no API permissions, with token automounting disabled. Sharing the namespace `default` account makes API activity unattributable; separate identities are free and make any future grant precise.

Migration path. Because the gateway owns the public contract, routes can move behind it one at a time — the strangler-fig pattern (Fowler, 2004) — so the existing deployment keeps serving while capabilities are cut over. That is why the gateway is a distinct workload, not ingress annotations.

Operational and security boundaries. Two decisions carry most of the resilience argument. First, `checkout-fn` applies a 1500 ms timeout budget per dependency, so a slow dependency cannot hold customer requests open. Second, dependencies are classified: pricing is **critical** (no total, no order), inventory **non-critical** (the order can be accepted with stock unconfirmed). Health probes follow the same logic (Kubernetes, 2026a). Liveness is shallow everywhere, because a liveness probe touching a dependency converts a dependency outage into a restart storm; among the application services only `inventory-fn`, which owns the datastore, probes it in readiness. `checkout-fn` readiness excludes its dependencies, since probing them would remove every replica and turn partial failure into total outage.

Figure 1 — Implemented architecture with trust tiers, workload identity and enforced policy. Solid arrows are permitted paths; dotted `DENIED` / `EGRESS DENIED` edges are refused by NetworkPolicy and are the cases verified in Appendix D.



2. Implementation Summary

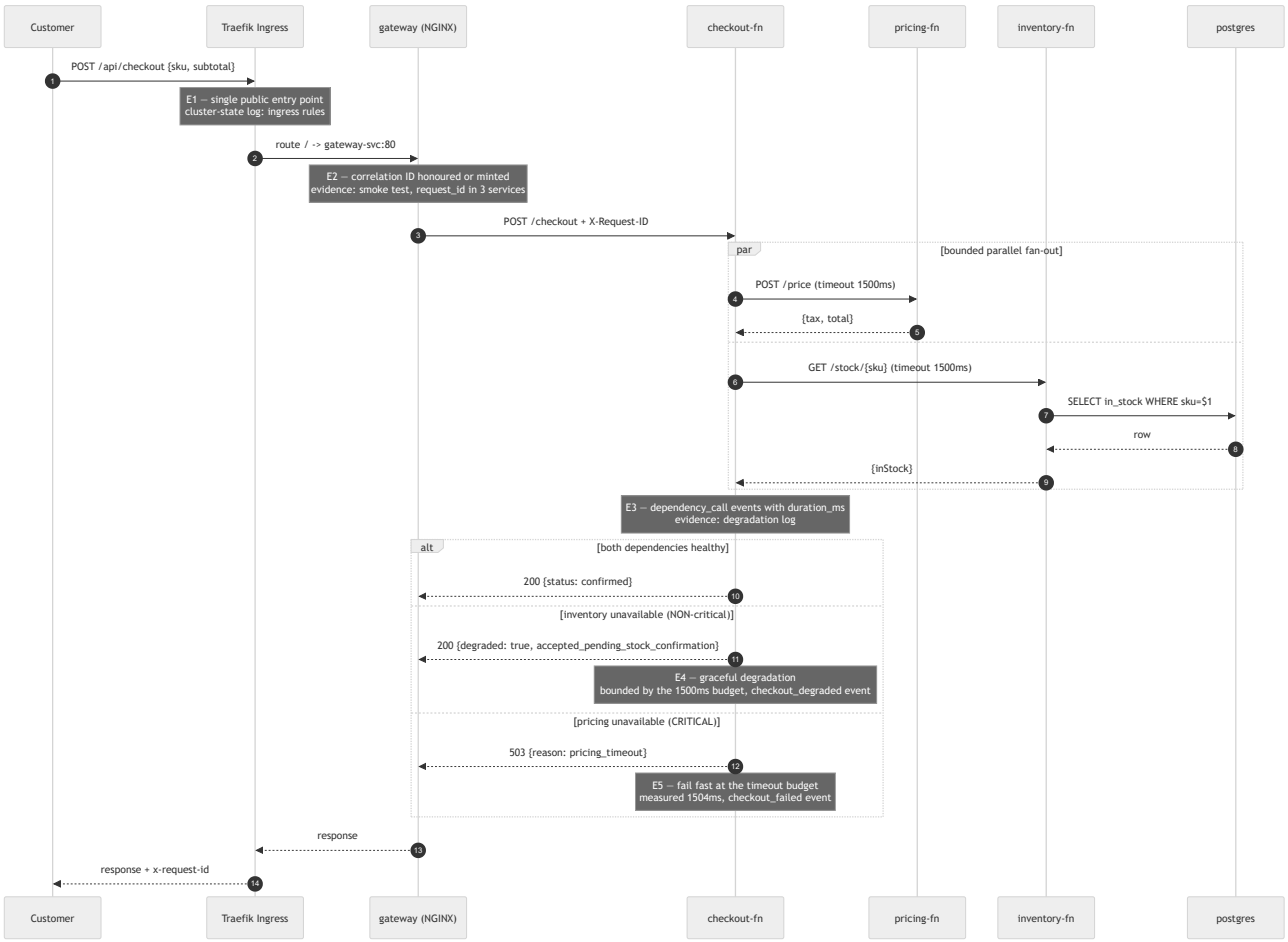
The prototype runs on single-node K3s (v1.36.3) on Ubuntu 24.04 (Rancher, 2026), installed with the module's procedure. Images are built locally and imported into containerd, so no registry is assumed.

Implemented: four Deployments (gateway ×2, checkout ×2, pricing ×2, inventory ×2) plus PostgreSQL; five ClusterIP Services; one Ingress carrying an edge rate-limit middleware; two ConfigMaps; one Secret; one PVC; twelve NetworkPolicies covering both directions; probes, resource limits and `restricted` Pod Security (Kubernetes, 2026c) on every workload, with non-root execution, dropped capabilities and no token mounting. Four choices map design intent onto configuration. Two replicas everywhere make the readiness gate meaningful: a bad replica can be withheld while another serves. `TIMEOUT_MS=1500` bounds a slow dependency. PostgreSQL uses `Recreate` because one RWO volume cannot be shared by two pods mid-rollout. The gateway runs `nginx-unprivileged` because stock NGINX cannot satisfy the `restricted` standard.

Structured JSON logging with request-ID propagation was added to the three application services. The platform deploys with one command and was verified by deleting the namespace and rebuilding: teardown 61 seconds, rebuild to a working checkout **30 seconds** (Appendix E).

Gaps against the proposal. Three things are designed but not implemented, for scope not oversight: the Ingress serves HTTP (no certificate authority in scope), the database is a single replica with no backup or failover, and the checkout API has no authentication. Each returns in Section 5.

Figure 2 — Main request flow and the evidence point behind each claim.



3. Operational Behaviour

The behaviour selected is **partial failure: graceful degradation versus fail fast**, because it tests the dependency classification directly. Healthy, a checkout returns `200 confirmed` in 11 ms. With `inventory-fn` scaled to zero it returns 200, marked degraded, in 1502 ms. With `pricing-fn` delayed 3000 ms it returns `503 pricing_timeout` in 1504 ms (Appendix B). A repeat run degraded in 8.5 ms, because a Service with no endpoints refused the connection rather than hanging: the budget bounds the cost of degradation without describing it.

Both claims hold: the non-critical dependency degrades rather than rejecting the customer, and the critical one fails fast at the configured budget. The measured 1502/1504 ms confirms the 1500 ms timeout is the binding constraint, not an accident of network timing. The cost is visible too: when the budget binds, degradation turns an 11 ms response into a 1502 ms one. Under sustained failure that consumes connection capacity (Google, 2016); a circuit breaker is the correct next step (Fowler, 2014).

A second experiment came from a real defect: an intermittent 502 during a rollout, reproduced by the control group. A `preStop` delay plus a 30-second grace period lets the endpoints controller remove the pod first (Kubernetes, 2026e). Run as a controlled A/B rather than a single run, the control failed 6 of 179 requests (3.4%) and the fixed group 3 of 208 (1.4%). At these sample sizes that difference is not statistically significant, and the control also ran a 1-second grace period against the 30-second default, so two variables moved together. What it establishes is that failures were **not eliminated**, and that an earlier run returning 204/204 was unrepresentative (Appendix C2). A third experiment tests admission control, which is load shedding at the edge. A 30-request burst without the rate-limit middleware (Traefik Labs, 2026) admitted all 30 at a p95 of 0.168 s; with it, 11 were admitted and 19 refused with HTTP 429, and the admitted requests completed at a p95 of 0.075 s. The two samples differ in size and selection, so this shows the direction of the effect rather than its magnitude (Appendix C1): admitting everything degrades everyone, whereas early rejection keeps the backend out of contention.

A fourth set asked what happens when a *change* is wrong rather than a dependency slow. A broken readiness path, a broken image tag, and the gateway upstream reverted to its Compose name were injected separately (Appendix C3–C5). All three were contained by the readiness gate a replica must pass before receiving traffic, and user-visible impact was nil: 10/10, 15/15 and 5/5 checkouts returned 200. Operators saw the failure; users did not, so a broken release can sit unnoticed unless rollout state is alerted on.

4. Observability and Security Validation

Observability. The three application services emit structured JSON with a shared `request_id`, and `checkout-fn` adds per-dependency `duration_ms`. One request is traceable across all three, and the per-hop timings identified the timeout as the binding constraint in Section 3. This is deliberately log-based rather than a full tracing stack: on one node, correlation IDs deliver most of the diagnostic value far more cheaply (Appendix A). The limitation is real: no sampling, retention, aggregation or dashboard.

Security validation 1 — are the trust boundaries real? The tiers were tested, not asserted: four documented paths must work, six lateral and five outbound paths must fail, and an unlabelled pod simulating a compromised workload must reach nothing. **All 19 cases passed** (Appendix D). The egress cases matter most: no application workload reaches the public internet, and the metadata endpoint at 169.254.169.254 is unreachable from the service holding database credentials. NetworkPolicy objects are silently inert without a policy-capable CNI, so the negative controls prove enforcement rather than intent. DNS is the one exception; without it service discovery fails and the outage looks like an application bug.

The limits matter as much as the result. The 19 cases cover four workloads, not Postgres; node traffic is always permitted regardless of policy; and NetworkPolicy records nothing about what it allowed or blocked, so this control has no audit trail. It isolates reachability, nothing more.

Security validation 2 — configuration and image posture. `kubesec` (Controlplane, 2026) scores all five Deployments, read from the live cluster so the score describes what actually runs. The four application workloads score **12** (a raw score, not a ratio) — non-root, dropped capabilities, read-only root filesystem, resource limits, dedicated identities, no token automounting. Postgres scores **11**, missing only `ReadOnlyRootFilesystem`. Trivy (Aqua Security, 2026) reported 1 CRITICAL and 6 HIGH per service image, including CVE-2026-59873 in `node-tar`. Interpretation mattered more than the count: every finding came from the npm CLI in the base image, not application dependencies, and was unreachable at runtime. Rather than suppress them, the images were rebuilt multi-stage without the package manager; the same scan returns **0 CRITICAL and 0 HIGH** (Appendix F).

Prioritised issues, ordered by how easily each is exploited without credentials. (1) Secrets are base64 only and K3s disables encryption at rest (Kubernetes, 2026d): any principal with `get secret` reads the database

password. (2) No authentication on the checkout API. (3) No TLS. (4) Lower priority: no AppArmor profile and UIDs below 10000. One flag is a false negative: kubesec reports missing seccomp, but its selector reads the deprecated annotation, while every workload sets `seccompProfile: RuntimeDefault` (Appendix F): taking the scanner literally would have meant fixing a control already in place.

5. Critical Evaluation

The strongest aspect is that the architecture's claims are testable and were tested: tier separation, degradation, timeout budget, persistence and rollout safety each have a script producing evidence. One script found a real defect; the control group measuring the fix showed it is partial. The weaknesses are equally clear. **Single points of failure**: one node, one database replica, so node loss is total outage; RPO and RTO are undefined and backups do not exist. **Identity is missing**: NetworkPolicy enforces reachability, not identity, so the trust model is entirely network-based, contradicting the Zero Trust principle that network location must not imply trust (NIST, 2020); a compromised `checkout-fn` is indistinguishable from a legitimate one to `inventory-fn`. mTLS with workload identity is the most valuable next increment. **Degradation is timeout-bound, not circuit-broken**. **The PVC gives restart survival, not recoverability**: it outlived pod replacement but died with the namespace during the rebuild (Appendix G).

Two trade-offs were accepted deliberately. A service mesh would supply mTLS, retries and telemetry uniformly, but for four services it adds a control plane and sidecar overhead this organisation cannot operate; NetworkPolicy plus application timeouts covers most of the benefit far more cheaply. Scale-to-zero was rejected because cold starts would add latency to the checkout path and idle cost is not the constraint at this size. A third alternative, asynchronous messaging, is notsettled. The two dependencies are called in parallel, so tail latency is bounded by the slower rather than their sum; what is still coupled is availability. The degraded response returned when inventory is unavailable is a promise to confirm stock later, and nothing records or honours it. A durable queue with idempotent consumers and a dead-letter path is the standard remedy, rejected because a stateful broker would add a second single point of failure on one node. The promise remains outstanding: an acknowledged gap, not a resolved trade-off.

If continued: encryption at rest and a secrets manager; TLS and authentication at the edge; workload identity for east–west calls; a circuit breaker on inventory; then backup and restore rehearsal, because untested recovery is an assumption, not a control.

6. References

Aqua Security (2026) *Trivy documentation*. Available at: https://trivy.dev/ (Accessed: 14 August 2026).

Controlplane (2026) *kubesec — security risk analysis for Kubernetes resources*. Available at: https://kubesec.io/ (Accessed: 14 August 2026).

Fowler, M. (2004) *StranglerFigApplication*. Available at: https://martinfowler.com/bliki/StranglerFigApplication.html (Accessed: 14 August 2026).

Fowler, M. (2014) *CircuitBreaker*. Available at: https://martinfowler.com/bliki/CircuitBreaker.html (Accessed: 14 August 2026).

Google (2016) *Site Reliability Engineering: Addressing Cascading Failures*. Available at: https://sre.google/sre-book/addressing-cascading-failures/ (Accessed: 14 August 2026).

Kubernetes (2026a) *Configure Liveness, Readiness and Startup Probes*. Available at: https://kubernetes.io/docs/tasks/configure-pod-container/configure-liveness-readiness-startup-probes/ (Accessed: 14 August 2026).

Kubernetes (2026b) *Network Policies*. Available at: https://kubernetes.io/docs/concepts/services-networking/network-policies/ (Accessed: 14 August 2026).

Kubernetes (2026c) *Pod Security Standards*. Available at: https://kubernetes.io/docs/concepts/security/pod-security-standards/ (Accessed: 14 August 2026).

Kubernetes (2026d) *Encrypting Confidential Data at Rest*. Available at: https://kubernetes.io/docs/tasks/administer-cluster/encrypt-data/ (Accessed: 14 August 2026).

Kubernetes (2026e) *Pod Lifecycle: termination of Pods*. Available at: https://kubernetes.io/docs/concepts/workloads/pods/pod-lifecycle/ (Accessed: 14 August 2026).

Lee, E. (2026) *Enterprise Architecture Design: Lectures 1–12 and Laboratory Materials*. ENTPH6001, TU Dublin.

NIST (2020) *SP 800-207: Zero Trust Architecture*. doi: 10.6028/NIST.SP.800-207.

Rancher (2026) *K3s documentation: Networking and Traefik Ingress*. Available at: https://docs.k3s.io/networking (Accessed: 14 August 2026).

Traefik Labs (2026) *Traefik Kubernetes Ingress documentation*. Available at: https://doc.traefik.io/traefik/providers/kubernetes-ingress/ (Accessed: 14 August 2026).

AI tools declaration

GitHub Copilot (Claude Opus 5) was used to assist with drafting manifests, service code, test scripts and report structure. All deployments, experiments and measurements reported here were executed by the author on the cluster described, and every figure quoted is taken from the captured evidence logs in the appendices. No result in this report is estimated or reproduced from memory.

Appendices — Evidence

Every figure quoted in the report comes from one of the log files in `submission/evidence/`. All were produced by the scripts in `submission/scripts/` against the live single-node K3s cluster. Each appendix names the source file so any claim can be traced to its raw output.

Re-running everything from nothing:

```
bash scripts/deploy-all.sh          # build, deploy, wait for readiness
bash scripts/30-smoke-test.sh       # Appendix A
bash scripts/45-test-rolling-update-ab.sh   # Appendix C2
bash scripts/50-test-degradation.sh # Appendix B
bash scripts/55-test-ratelimit.sh   # Appendix C1
bash scripts/56-test-readiness-gate.sh
bash scripts/57-test-rollout-failure-recovery.sh
bash scripts/58-test-misconfig-diagnosis.sh
bash scripts/59-test-scaling.sh
bash scripts/60-test-networkpolicy.sh
bash scripts/70-test-persistence.sh
bash scripts/80-security-audit.sh
bash scripts/81-trivy-v1-vs-v2.sh
bash scripts/82-trivy-v1-detail.sh          # Appendix F, CVE attribution
bash scripts/83-kubesec-all-and-metrics.sh  # Appendix F, per-workload scores
bash scripts/90-capture-evidence.sh         # Appendix A, cluster state
bash scripts/95-rebuild-from-scratch.sh
```

Appendix A — Platform state and the main request path

Source: `cluster-state-*.log`, `smoke-test-*.log`, `scaling-*.log` (node capacity), `rebuild-from-scratch-*.log` (object inventory)

Cluster: K3s v1.36.3+k3s1 on Ubuntu 24.04.4 LTS, single node, 2 vCPU / 7.8 GB. Namespace `shop` with `pod-security.kubernetes.io/enforce: restricted`.

Deployed objects: 5 Deployments (gateway ×2, checkout-fn ×2, pricing-fn ×2, inventory-fn ×2, postgres ×1), 5 ClusterIP Services, 1 Ingress, 2 ConfigMaps, 1 Secret, 1 PVC, 5 ServiceAccounts, 12 NetworkPolicies, 1 Traefik Middleware.

A successful checkout, showing the composed result:

```
{"status":"confirmed","degraded":false,
 "pricing":{"subtotal":100,"taxRate":0.23,"tax":23,"total":123},
 "stock":{"sku":1,"inStock":true,"confirmed":true},
 "requestId":"..."}
```

Request correlation across three services. One request ID appears in all three service logs, which is what makes a distributed request diagnosable. The log lines are JSON; the fields relevant to correlation are extracted here from `smoke-test-20260814-161057.log`, with every value unchanged:

```
checkout-fn   request_id=smoke-1786723857 dependency=inventory status=200 duration_ms=98.1
checkout-fn   request_id=smoke-1786723857 dependency=pricing   status=200 duration_ms=299.3
checkout-fn   request_id=smoke-1786723857 route=/checkout      status=200 duration_ms=301.8
pricing-fn    request_id=smoke-1786723857 route=/price         status=200 duration_ms=0.6
inventory-fn  request_id=smoke-1786723857 route=/stock/1       status=200 duration_ms=3.3
```

These are first-request timings taken immediately after a full rebuild, so they include Node.js process warm-up and the initial database connection: `checkout-fn` records 299.3 ms for the pricing call while `pricing-fn` records 0.6 ms of actual work. That gap is the cost of a cold start, not of the dependency, and it disappears

on subsequent requests — warm end-to-end checkouts elsewhere in the evidence run from 11 ms to 46 ms. The discrepancy is left visible here because it is exactly what per-hop timings are for: without them the delay would have been misattributed to `pricing-fn`.

Input validation is enforced at the edge of the service: a negative subtotal returns HTTP 400 rather than being priced.

Appendix B — Degradation under dependency failure

Source: `degradation-20260814-121529.log` (B1–B5), `degradation-20260818-174328.log` (repeat run)

Stage	Condition	Result	Latency
B1	healthy	HTTP 200, `degraded:false`	11 ms
B2	`inventory-fn` scaled to 0	HTTP 200, `status:accepted_pending_stock_confirmation`	1502 ms
B3	`inventory-fn` restored	HTTP 200, `degraded:false`	14 ms
B4	`pricing-fn` delayed 3000 ms	HTTP 503	1504 ms
B5	delay removed	HTTP 200	46 ms

Latencies in B2 and B4 are the service-side `duration_ms`; B1, B3 and B5 are end-to-end times measured at `curl` (0.010579 s, 0.013624 s and 0.045857 s, rounded), which for B2 and B4 were 1504.6 ms and 1510.1 ms. Both sit on the 1500 ms timeout budget, which is the intended behaviour: a failing dependency costs the budget and no more.

The asymmetry between B2 and B4 is the dependency classification working — inventory is non-critical and degrades, pricing is critical and fails closed.

The degraded case is not a single number, and the repeat run is kept because it disagrees. On 18 August the same test degraded in 8.5 ms rather than 1502 ms:

```
{"event":"dependency_error","request_id":"deg-inventory-down","dependency":"inventory",
 "error":"unreachable","duration_ms":3}
HTTP 200  total=0.008458s
```

With no endpoints behind `inventory-svc` the connection was refused immediately instead of hanging until the budget expired. Both outcomes are correct; which one occurs depends on whether the dead dependency refuses or silently drops. The timeout is therefore a **bound on** the cost of degradation, not a measurement of it — 1502 ms is the worst case, not the typical case. The B4 critical-path figure is unaffected, because there the dependency answers slowly rather than not at all, so the budget always binds.

The repeat run also carries the per-pod counters that the August 14 run left empty (`requests_total`, `errors_total`, `dependency_failures_total`, `degraded_responses_total`); only the pod that served the injected failures shows `errors_total 1` and `degraded_responses_total 1`, which is consistent with two replicas sharing traffic.

Appendix C — Operational behaviour experiments

C1 — Edge admission control

Source: `ratelimit-*.log`. 30 concurrent requests, with and without the Traefik middleware (`average: 5, burst: 10`).

Condition	Admitted	Rejected	p95 of admitted
No rate limit	30	0	0.168 s
Rate limited	11	19 × HTTP 429	0.075 s

C2 — Rolling update, controlled A/B

Source: `rolling-update-ab-*.log`. Continuous requests during a gateway image change.

Group	Configuration	Requests	Failures	Rate
A (control)	no `preStop`, 1 s grace	179	6	3.4%
B (fix)	`preStop` 5 s, 30 s grace	208	3	1.4%

Failures were recorded as HTTP 502 or as `000`, the latter meaning `curl` could not complete the transfer. `000` is not asserted to be a timeout specifically.

`rolling-update-gateway-*.log` is an earlier single run of the same change that returned 204/204 successes. It is retained deliberately: it is the run that would have supported a stronger claim than the evidence justifies, and the A/B experiment is what showed it to be unrepresentative.

C3 — Bad readiness probe: Running but not Ready

Source: `readiness-gate-*.log`. The readiness path was pointed at `/definitely-not-a-real-path`.

```
pricing-fn-58d5cf59d6-phdmf   10.42.0.86   false   Running
pricing-fn-84ddfd689b-c4592   10.42.0.67   true    Running
pricing-fn-84ddfd689b-dgzdk   10.42.0.72   true    Running

not-Ready pod IP : 10.42.0.86
endpoint IPs     : [10.42.0.67 10.42.0.72]
Readiness probe failed: HTTP probe failed with statuscode: 404
```

The not-Ready pod's IP is absent from the Service endpoints, the rollout stalls at "1 out of 2 new replicas have been updated", and 10/10 checkouts still return 200.

C4 — Bad image tag and rollback

Source: `rollout-failure-recovery-*.log`, quoted verbatim.

```
    exit code from rollout status: 124  (non-zero = the platform refused to finish the
release)
    checkout-fn-5949488f54-kxs2g   1/1   Running        0     40m
    checkout-fn-5949488f54-tqmzp   1/1   Running        0     40m
    checkout-fn-699f66c9db-wjvk6   0/1   ErrImagePull   0     65s
      checkout-fn-5949488f54  DESIRED=2  CURRENT=2  READY=2
      checkout-fn-699f66c9db  DESIRED=1  CURRENT=1  READY=0
      checkout-fn-76b747ff7d  DESIRED=0  CURRENT=0  READY=0
```

The third ReplicaSet is an earlier revision already scaled to zero. 15/15 checkouts returned 200 throughout. `kubectl rollout undo` restored the previous revision in under one second (`rollback completed in 0s`).

C5 — Configuration error diagnosis (Lab 4.2, required exercise)

Source: `misconfig-diagnosis-*.log`, quoted verbatim. The gateway upstream was changed to the Compose service name.

```
nginx: [emerg] host not found in upstream "checkout-fn" in /etc/nginx/conf.d/default.conf:17
       gateway-6dbd8ff985-2v8n5   0/1   CrashLoopBackOff   3 (30s ago)   67s
```

Diagnosis path: gateway pods → no Service named `checkout-fn` → the real Service `checkout-svc` has healthy endpoints → the gateway's own log names the cause. The backend was never unhealthy.

The lab predicts HTTP 503. This platform returned 200 throughout, because the broken pod never passed readiness and the previous replicas kept serving. The script asserts that the injection actually took effect before reporting, after a first run silently failed to inject and produced a misleading "healthy" log.

C6 — Manual scaling

Source: `scaling-*.log`. `checkout-fn` scaled 2 → 10 → 2; 14/14 requests returned 200 throughout, 10 replicas Ready in 10 s, no Pending pods. Node reservation at 10 replicas: CPU requests 55%, CPU limits 254% — the limits are oversubscribed, which is acceptable only because these services are latency-bound rather than CPU-bound.

Appendix D — Trust boundary enforcement

Source: `networkpolicy-*.log`. 19 cases, all passing.

Group	Cases	Expectation
A. Documented paths	gateway→checkout, checkout→pricing, checkout→inventory, inventory→postgres	ALLOWED
B. Lateral movement	gateway→pricing, gateway→inventory, gateway→postgres, pricing→inventory, pricing→postgres, checkout→postgres	BLOCKED
C. Compromised workload (unlabelled pod)	→checkout, →pricing, →inventory, →postgres	BLOCKED
D. Egress	checkout→internet, pricing→internet, inventory→internet, gateway→internet, inventory→169.254.169.254	BLOCKED

D5 is the cloud metadata endpoint, reachable from the workload holding database credentials if egress were unrestricted. DNS is the single deliberate egress exception.

Appendix E — Rebuild from nothing

Source: `rebuild-from-scratch-*.log`. Fields are relabelled for readability; the values are the log's own (`real 1m0.747s`, `real 0m30.042s`, `HTTP 200`, `networkpolicies: 12`).

```
teardown (namespace delete)          real 1m0.747s
rebuild (deploy-all.sh)              real 0m30.042s
verify main path                     HTTP 200
networkpolicies after rebuild        12
```

The PVC is deleted with the namespace, so seeded stock data is recreated by `inventory-fn` on startup. This is the data-lifecycle coupling discussed in Section 5, and it is visible here rather than hidden.

Appendix F — Security posture

Source: `security-audit-*.log`, `trivy-v1-vs-v2-*.log`, `kubesec-all-workloads-*.log`

kubesec (raw score, higher is better; every Deployment read from the live cluster by `83-kubesec-all-and-metrics.sh`, so these describe what is actually running). The rule list is printed by the script itself rather than inferred:

Deployment	Score	Rules kubesec did not mark satisfied
`checkout-fn`	12	ApparmorAny, SeccompAny, RunAsGroup, RunAsUser
`gateway`	12	ApparmorAny, SeccompAny, RunAsGroup, RunAsUser
`inventory-fn`	12	ApparmorAny, SeccompAny, RunAsGroup, RunAsUser
`pricing-fn`	12	ApparmorAny, SeccompAny, RunAsGroup, RunAsUser
`postgres`	11	the same four, plus `ReadOnlyRootFilesystem`

Two of these need interpreting rather than fixing.

`ReadOnlyRootFilesystem` is the whole of the one-point gap for Postgres: it is set `false` in `20-postgres.yaml` and `true` in the four application workloads.

`SeccompAny` is a **false negative**. Its selector is the deprecated annotation `container.seccomp.security.alpha.kubernetes.io/pod`, while every workload sets the current field. The same log prints the API's own answer:

```
    checkout-fn seccompProfile=RuntimeDefault
    gateway seccompProfile=RuntimeDefault
    inventory-fn seccompProfile=RuntimeDefault
    postgres seccompProfile=RuntimeDefault
    pricing-fn seccompProfile=RuntimeDefault
```

The earlier `80-security-audit.sh` run scored only two workloads because piping a multi-document manifest to `kubesec scan /dev/stdin` reads the first document alone; script `83` was written to close that gap.

Workload identity. The same log lists every pod with the ServiceAccount it actually runs under, which is what turns "each service has its own identity" from a manifest claim into an observation:

```
  checkout-fn-5949488f54-5rj9c    checkout-sa
 checkout-fn-5949488f54-t67dd    checkout-sa
 gateway-79744d5f8f-8mz8j        gateway-sa
 gateway-79744d5f8f-vssk2        gateway-sa
 inventory-fn-548b5bfb66-5brhf   inventory-sa
 inventory-fn-548b5bfb66-w8kr7   inventory-sa
 postgres-8bf8f9f9b-s8glh        postgres-sa
 pricing-fn-84ddfd689b-hwg9d     pricing-sa
 pricing-fn-84ddfd689b-wb5vt     pricing-sa
 pods using the default ServiceAccount: 0
```

Trivy, same scan against both image generations (`81-trivy-v1-vs-v2.sh`):

Image	CRITICAL	HIGH
`ead/pricing-fn:v1`	1	6
`ead/inventory-fn:v1`	1	6
`ead/checkout-fn:v1`	1	6
`ead/pricing-fn:v2`	0	0
`ead/inventory-fn:v2`	0	0
`ead/checkout-fn:v2`	0	0

The cluster runs the `v2` images. `82-trivy-v1-detail.sh` retains the finding-level detail behind the interpretation (`trivy-v1-detail-*.log`): `Total: 7 (HIGH: 6, CRITICAL: 1)`, the CRITICAL being `CVE-2026-59873` in `tar` (node-tar, gzip-bomb denial of service), alongside `brace-expansion`, `ip-address` and `undici` findings. The same script inspects both images directly:

```
 npm CLI present in v1
npm CLI absent in v2
```

That is the evidence for the claim that the findings came from the bundled package manager rather than from anything the service loads at runtime, and that removing it is a real fix rather than a suppression.

Workload identity: 9/9 pods run under a dedicated ServiceAccount and none uses `default`, as listed pod by pod above from `kubesec-all-workloads-*.log`; token automounting is disabled on all nine, per section 3.3 of `security-audit-*.log`.

Repository credential scan: PASS — no literal credentials in manifests or source. The credential is generated at deploy time by `20-create-secret.sh`; the repository holds only `11-secret.example.yaml` with a placeholder.

Disclosed residual risk: the audit reports that `PGPASSWORD` is retrievable as base64 by any principal holding `get secret`, and that K3s stores Secrets unencrypted at rest by default. The audit script reports the

credential's length only and never its value.

Appendix G — Data lifecycle

Source: `persistence-*.log`. A marker row was written, the PostgreSQL pod deleted, and the marker read back from the replacement pod:

```
 looked for: marker-1786723909
found     : marker-1786723909
RESULT: PASS - data survived pod replacement
```